1. ការណែនាំអំពីការកម្មវិធី C

កម្មវិធី C គឺជាភាសាកម្មវិធីទូទៅដែលមានអំណាច។ វាត្រូវបានប្រើប្រាស់យ៉ាងទូលំទូលាយក្នុងសoftware ប្រព័ន្ធ និងកម្មវិធីអនុគមន៍, និងទីកន្លែងផ្សេងទៀតដែលការអនុវត្តមានសារសំខាន់។

1.1. សេចក្តីពិពណ៌នាអំពី C

C គឺជាភាសាកម្មវិធីដែលមានភាពច្រើនប្រើ និងមានការចាប់ផ្តើមសំខាន់សម្រាប់ភាសាកម្មវិធីទំនើបផ្សេងៗដូចជា C++, Java និង Python។ ភាសានេះមានការត្រួតពិនិត្យលើប្រព័ន្ធធនធាន។

1.2. ប្រវត្តិសាស្ត្រ C

C ត្រូវបានបង្កើតដោយ Dennis Ritchie នៅឆ្នាំ 1972 នៅ Bell Labs។ វាត្រូវបានរចនាឡើងសម្រាប់ការសរសេរប្រព័ន្ធប្រតិបត្តិការ និងបានក្លាយជាភាសាកម្មវិធីមួយដែលមានប្រជាប្រិយភាពខ្ពស់។

គន្លឹះសំខាន់ក្នុងប្រវត្តិសាស្ត្រ C:

- 1972: ការបង្កើត C នៅ Bell Labs
- 1978: ការបោះពុម្ព "The C Programming Language" ដោយ Kernighan និង Ritchie
- 1989: ការចុះបញ្ជីស្តង់ដារអេសអេនអេស C (ANSI C)

1.3. លក្ខណៈពិសេសរបស់ C

C មានលក្ខណៈពិសេសមួយចំនួនដែលធ្វើឲ្យវាជាជម្រើសមួយដែលពេញនិយមសម្រាប់កម្មវិធីជាច្រើន៖

- ការចូលប្រើការជ្រៅទៅកាន់សមត្ថភាពអង្គចងចាំ
- វាតាមរយៈSyntax និងសំណុំរចនាបថដែលសាមញ្ញ
- អាចប្រើបានលើវេទិកានានា និងអាចដំណើរការបាននៅលើវេទិកាផ្សេងៗ
- បណ្ណាល័យសំណុំការងារដែលមានសម្បត្តិ
- ការប្រើប្រាស់ធនធានយ៉ាងមានប្រសិទ្ធភាព

1.4. ការរៀបចំបរិស្ថាន C

ដើម្បីចាប់ផ្តើមការសរសេរកម្មវិធីនៅក្នុង C អ្នកត្រូវតែចាប់ផ្តើមបរិស្ថានអភិវឌ្ឍ:

- តំឡើងកម្មវិធីចម្រោះ C (ឧ. GCC ឬ Clang)
- ជ្រើសរើសប្រព័ន្ធអភិវឌ្ឍន៍ឯកសារ (IDE) ដូចជា Code::Blocks, Dev-C++ ឬ Visual Studio
- សរសេរកម្មវិធី C យ៉ាងសាមញ្ញនៅក្នុងកម្មវិធីអភិវឌ្ឍ
- ចម្រោះ និងអនុវត្តកម្មវិធី

```c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

1.5. រចនាសម្ព័ន្ធកម្មវិធី C

កម្មវិធី C មួយធម្មតាមានរចនាសម្ព័ន្ធដូចខាងក្រោម:

```c
#include <stdio.h>

// ព្រឹត្តិការណ៍ Preprocessor
int main() {
    // មុខងារជាMain
    // ការបញ្ជាក់អថេរ
    int number = 10;

    // អនុគមន៍
    printf("Number: %d", number);

    return 0; // ពិពណ៌នាអត្រាប្រតិបត្តិការ
}
```

ការពន្យល់:

- **ព្រឹត្តិការណ៍ Preprocessor:** បញ្ចូលបណ្ណាល័យដែលចាំបាច់
- **មុខងារMain:** ចំណុចចាប់ផ្តើមនៃកម្មវិធី
- **ការបញ្ជាក់អថេរ:** ការបញ្ជាក់អថេរដែលបានប្រើនៅក្នុងកម្មវិធី
- **អនុគមន៍:** ប្រតិបត្តិការកម្មវិធី
- **ពិពណ៌នាអត្រាប្រតិបត្តិការ:** បញ្ចប់កម្មវិធី

2. អថេរ និង ប្រភេទទិន្នន័យ

ផ្នែកនេះគួរឲ្យដឹងពីមូលដ្ឋាននៃអថេរ និងប្រភេទទិន្នន័យនៅក្នុងការសរសេរប្រព័ន្ធ C ។

2.1. ការប្រកាសអថេរ

នៅក្នុង C អថេរត្រូវបានប្រើដើម្បីផ្ទុកទិន្នន័យ។ ដើម្បីប្រកាសអថេរ អ្នកត្រូវតែបញ្ជាក់ប្រភេទ និងឈ្មោះរបស់វា។ ឧទាហរណ៍៖

```
int number;
float pi;
char letter;
```

នៅទីនេះ 'int' ត្រូវបានប្រើសម្រាប់ចំនួនគត់ 'float' សម្រាប់ចំនួនបែបភាគរយ និង 'char' សម្រាប់តួអក្សរ។

2.2. ប្រភេទទិន្នន័យមូលដ្ឋាន

C មានប្រភេទទិន្នន័យមូលដ្ឋានជាច្រើន រួមមាន៖

- **int**: ប្រើសម្រាប់ចំនួនគត់ (ឧ. 10, -50)
- **float**: ប្រើសម្រាប់ចំនួនបែបភាគរយ (ឧ. 3.14, -0.001)
- **double**: ប្រើសម្រាប់ចំនួនបែបភាគរយដែលមានភាពត្រឹមត្រូវខ្ពស់ (ឧ. 3.14159265359)
- **char**: ប្រើសម្រាប់តួអក្សរ (ឧ. 'a', 'Z')

2.3. តម្លៃថេរ

តម្លៃថេរនេះគឺជាតម្លៃដែលមិនអាចផ្លាស់ប្តូរបានក្នុងការប្រតិបត្តិការនៃកម្មវិធី។ វាត្រូវបានកំណត់ដោយប្រើគន្លង `#define` ឬក៏ប្រើកូនសោ `const`៖

```
#define PI 3.14159
const int MAX_VALUE = 100;
```

ឧទាហរណ៍ទីមួយកំណត់តម្លៃថេរដើម្បីសម្គាល់ PI ហើយឧទាហរណ៍ទីពីរប្រើកូនសោ `const` សម្រាប់តម្លៃថេរចំនួនគត់។

2.4. ការបម្លែងប្រភេទ និង ការបម្លែងទិន្នន័យ

ការបម្លែងប្រភេទគឺជាដំណើរការនៃការបម្លែងប្រភេទទិន្នន័យពីមួយទៅមួយ។ នៅក្នុង C អ្នកអាចធ្វើបានទាំងដោយស្វ័យប្រវត្តិក្នុងករណីណាមួយ ឬដោយគ្រូបង្រៀន៖

- **ការបម្លែងដោយស្វ័យប្រវត្តិ**: អ្នកកុំព្យូទ័រប្រែប្រភេទដោយស្វ័យប្រវត្តិពេលបម្លែងពីប្រភេទតូចទៅធំជាង ដូចជា ពី `int` ទៅ `float` ។
- **ការបម្លែងដោយដៃ**: អ្នកអភិវឌ្ឍន៍ធ្វើការបម្លែងដោយដៃ ដូចជា បម្លែងពី `float` ទៅ `int` ។

```
int num = 10;
float numFloat = num;  // ការបម្លែងដោយស្វ័យប្រវត្តិ
```

```
numFloat = 10.5;
int numInt = (int) numFloat;  // ការបម្លែងដោយដៃ
```

2.5. ប្រភេទផ្ទុកទិន្នន័យ

ប្រភេទផ្ទុកទិន្នន័យក្នុង C គឺកំណត់ព្រមទាំងវិស័យ ការច visibility និង អាយុរស់រវើកនៃអថេរ។ មានប្រភេទផ្ទុកទិន្នន័យបួនប្រភេទ៖

- **auto**: ប្រភេទផ្ទុកទិន្នន័យលំនាំសម្រាប់អថេរមូលដ្ឋាន។ ពួកវាត្រូវបានបង្កើតដោយស្វ័យប្រវត្តិនៅពេលហៅមុខងារ។
- **register**: ប្រើសម្រាប់ផ្ទុកអថេរនៅក្នុងចំណងចងក្រង CPU ជំនួសការផ្ទុកនៅក្នុង RAM ដើម្បីចូលរួមបានលឿន។
- **static**: ប្រើសម្រាប់រក្សាតម្លៃរបស់អថេរប្រសិនបើបន្តរក្សាតម្លៃនោះរហូតបន្ទាប់ពីការចាកចេញពីមុខងារ។
- **extern**: ប្រើសម្រាប់ប្រកាសអថេរដែលបានកំណត់ក្នុងឯកសារផ្សេង។

3. OPERATORS_ នៅក្នុង C

នៅក្នុងផ្នែកនេះ យើងនឹងស្វែងយល់ពីប្រភេទនៃ OPERATOR_ ក្នុងកម្មវិធី C ។

3.1. OPERATOR_គណិតវិទ្យា

OPERATOR_គណិតវិទ្យាត្រូវបានប្រើសម្រាប់ធ្វើការប្រតិបត្តិការគណិតវិទ្យា។ OPERATOR_គណិតវិទ្យាមូលដ្ឋានក្នុង C មានដូចតទៅ៖

- **+**: បូក
- **-**: បន្ថយ
- *****: គុណ
- **/**: ចែក
- **%**: បំបែកសល់ (បរិមាណនៅក្រោយការចែក)

ឧទាហរណ៍៖

```
int a = 10, b = 5;
int sum = a + b; // បូក
int product = a * b; // គុណ
```

3.2. OPERATOR_ប្រៀបធៀប

OPERATOR_ប្រៀបធៀបត្រូវបានប្រើសម្រាប់ប្រៀបធៀបតម្លៃពីរដើម្បីទទួលបានលទ្ធផល boolean (ពិត ឬ មិនពិត)។ OPERATOR_ទាំងនេះមានដូចតទៅ៖

- **==**: ស្មើនឹង
- **!=**: មិនស្មើនឹង
- **>**: លើស
- **<**: តិចជាង
- **>=**: លើសឬស្មើ
- **<=**: តិចឬស្មើ

ឧទាហរណ៍៖

```
int a = 10, b = 5;
bool result = (a > b); // ពិត ពីព្រោះ 10 លើស 5
```

3.3. OPERATOR_តុល្យភាព

OPERATOR_តុល្យភាពត្រូវបានប្រើសម្រាប់សម្ពោធន៍បញ្ចូលនៃការបញ្ជាក់លក្ខខណ្ឌ។ OPERATOR_ទាំងនេះមានដូចតទៅ៖

- **&&&**: AND តុល្យភាព
- **||**: OR តុល្យភាព
- **!**: NOT តុល្យភាព

ឧទាហរណ៍៖

```
int a = 10, b = 5;
bool result = (a > b && b > 0); // ពិត ប្រសិនបើលក្ខខណ្ឌទាំងពីរពិត
```

3.4. OPERATOR_ការបញ្ជាក់តម្លៃ

OPERATOR_ការបញ្ជាក់តម្លៃត្រូវបានប្រើសម្រាប់បញ្ជាក់តម្លៃទៅអថេរ។ OPERATOR_ការបញ្ជាក់មូលដ្ឋានគឺ៖

- **=**: ការបញ្ជាក់តម្លៃសាមញ្ញ
- **+=**: បូកនិងបញ្ជាក់
- **-=**: បន្ថយនិងបញ្ជាក់
- ***=**: គុណនិងបញ្ជាក់
- **/=**: ចែកនិងបញ្ជាក់
- **%=**: បំបែកសល់និងបញ្ជាក់

ឧទាហរណ៍៖

```
int a = 10;
a += 5; // a = a + 5
```

3.5. OPERATOR_បញ្ចូលជាតំណ

OPERATOR_បញ្ចូលជាតំណត្រូវបានប្រើសម្រាប់ធ្វើប្រតិបត្តិការលើប៊ីតនៃតម្លៃចំនួនគត់៖

- **&**: AND ប៊ីត
- **|**: OR ប៊ីត
- **^**: XOR ប៊ីត
- **~**: NOT ប៊ីត
- **<<**: ការផ្លាស់ទីឆ្វេង
- **>>**: ការផ្លាស់ទីស្តាំ

ឧទាហរណ៍៖

```
int a = 5, b = 3;
int result = a & b; // AND ប៊ីត
```

3.6. បន្ថែម និង ការបន្ថយ

OPERATOR_បន្ថែម និង OPERATOR_បន្ថយត្រូវបានប្រើសម្រាប់បង្កើន ឬ កាត់បន្ថយតម្លៃអថេរដោយ 1៖

- **++**: OPERATOR_បន្ថែម (បង្កើនតម្លៃដោយ 1)
- **--**: OPERATOR_បន្ថយ (កាត់បន្ថយតម្លៃដោយ 1)

ឧទាហរណ៍៖

```
int a = 5;
a++; // a ក្លាយជាលេខ 6
--a; // a ក្លាយជាលេខ 5 វិញ
```

3.7. OPERATOR_លក្ខខណ្ឌ

OPERATOR_លក្ខខណ្ឌ (OPERATOR_ ternary) គឺជាការបង្ហាញខ្លីសម្រាប់បញ្ជាក់ if-else៖

- **condition ? expr1 : expr2**: ប្រសិនបើលក្ខខណ្ឌពិត expr1 នឹងត្រូវបានអនុវត្ត។ ប្រសិនបើមិនពិត expr2 នឹងត្រូវបានអនុវត្ត។

ឧទាហរណ៍៖

```
int a = 5;
int result = (a > 3) ? 10 : 20; // result នឹងមានតម្លៃ 10 ពីព្រោះលក្ខខណ្ឌពិត
```

4. គ្រប់គ្រងប្រតិបត្តិការនៅក្នុង C

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីប្រភេទនៃប្រតិបត្តិការគ្រប់គ្រងនៅក្នុងភាសា C។

4.1. ប្រកាស if

ប្រកាស `if` ត្រូវបានប្រើដើម្បីធ្វើការត្រួតពិនិត្យលក្ខខណ្ឌមួយ។ ប្រសិនបើលក្ខខណ្ឌត្រឹមត្រូវ វានឹងអនុវត្តកូដក្នុងរយៈពេល។

```
#include <stdio.h>

int main() {
    int a = 10;
    if (a > 5) {
        printf("a is greater than 5");
    }
    return 0;
}
```

ការពន្យល់៖
- កម្មវិធីនេះពិនិត្យតម្លៃរបស់អថេរ a។
- ប្រសិនបើ a ធំជាង 5, វានឹងបង្ហាញ "a is greater than 5"។
- ប្រសិនបើលក្ខខណ្ឌមិនពិត, កូដនៅក្នុង `if` នឹងមិនត្រូវបានអនុវត្តទេ។

```
Output: a is greater than 5
```

4.2. ប្រកាស if-else

ប្រកាស `if-else` ត្រូវបានប្រើដើម្បីធ្វើការត្រួតពិនិត្យលក្ខខណ្ឌ។ ប្រសិនបើលក្ខខណ្ឌត្រឹមត្រូវ វានឹងអនុវត្តកូដក្នុងផ្នែក `if`។ បើមិនដូច្នេះទេ វានឹងអនុវត្តកូដក្នុងផ្នែក `else`។

```c
#include <stdio.h>

int main() {
    int a = 3;
    if (a > 5) {
        printf("a is greater than 5");
    } else {
        printf("a is not greater than 5");
    }
    return 0;
}
```

ការពន្យល់៖

- កម្មវិធីនេះពិនិត្យតម្លៃរបស់អថេរ a។
- ប្រសិនបើ a ធំជាង 5, វានឹងបង្ហាញ "a is greater than 5"។
- បើមិនដូច្នេះទេ, វានឹងបង្ហាញ "a is not greater than 5"។

```
Output: a is not greater than 5
```

4.3. ប្រកាស if-elseif-else

ប្រកាស `if-elseif-else` ត្រូវបានប្រើដើម្បីធ្វើការត្រួតពិនិត្យលក្ខខណ្ឌច្រើន។ វាអនុញ្ញាតឱ្យអ្នកពិនិត្យលក្ខខណ្ឌជាច្រើនក្នុងលំដាប់។

```c
#include <stdio.h>

int main() {
    int a = 10;
    if (a > 15) {
        printf("a is greater than 15");
    } else if (a > 10) {
        printf("a is greater than 10 but less than or equal to 15");
    } else {
        printf("a is less than or equal to 10");
    }
    return 0;
}
```

ការពន្យល់៖
- កម្មវិធីនេះពិនិត្យតម្លៃរបស់អថេរ a។
- ប្រសិនបើ a ធំជាង 15, វានឹងបង្ហាញ "a is greater than 15"។
- ប្រសិនបើ a ធំជាង 10 ប៉ុន្តែតូចជាងឬស្មើ 15, វានឹងបង្ហាញ "a is greater than 10 but less than or equal to 15"។
- បើមិនដូច្នេះទេ, វានឹងបង្ហាញ "a is less than or equal to 10"។

```
Output: a is less than or equal to 10
```

4.4. ប្រកាស switch

ប្រកាស switch ត្រូវបានប្រើដើម្បីធ្វើការត្រួតពិនិត្យលក្ខខណ្ឌច្រើន។ វាអនុញ្ញាតឱ្យអ្នកពិនិត្យតម្លៃរបស់អថេរមួយប្រឆាំងនឹងតម្លៃជាច្រើន។

```c
#include <stdio.h>

int main() {
    int day = 3;
    switch (day) {
        case 1:
            printf("Monday");
            break;
        case 2:
            printf("Tuesday");
            break;
        case 3:
            printf("Wednesday");
            break;
        default:
            printf("Invalid day");
    }
    return 0;
}
```

ការពន្យល់៖
- កម្មវិធីនេះពិនិត្យតម្លៃរបស់អថេរ day។
- ប្រសិនបើ day ស្មើនឹង 1, វានឹងបង្ហាញ "Monday"។
- ប្រសិនបើ day ស្មើនឹង 2, វានឹងបង្ហាញ "Tuesday"។
- ប្រសិនបើ day ស្មើនឹង 3, វានឹងបង្ហាញ "Wednesday"។
- បើមិនដូច្នេះទេ, វានឹងបង្ហាញ "Invalid day"។

```
Output: Wednesday
```

4.5. ប្រកាស break

ប្រកាស break ត្រូវបានប្រើដើម្បីបញ្ចប់ការអនុវត្តនៃកូដក្នុង switch ឬ loop។

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            break; // បញ្ចប់ loop នៅពេល i ស្មើ 3
        }
        printf("%d", i);
    }
    return 0;
}
```

ការពន្យល់៖
- កម្មវិធីនេះប្រើ break ដើម្បីបញ្ចប់ for loop នៅពេល i ស្មើ 3។
- ដូច្នេះ, វានឹងបង្ហាញតែលេខ 1 និង 2។

```
Output: 1
2
```

4.6. ប្រកាស continue

ប្រកាស continue ត្រូវបានប្រើដើម្បីរំលងការអនុវត្តនៃកូដក្នុង loop និងបន្តទៅការប្រតិបត្តិការបន្ទាប់។

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue; // រំលងការប្រតិបត្តិការនៅពេល i ស្មើ 3
        }
        printf("%d", i);
    }
```

```
    return 0;
}
```

ការពន្យល់៖
- កម្មវិធីនេះប្រើ `continue` ដើម្បីរំលងការប្រតិបត្តិការនៅពេល `i` ស្មើ 3។
- ដូច្នេះ, វានឹងបង្ហាញលេខ 1, 2, 4, និង 5។

```
Output: 1
2
4
5
```

4.7. ប្រកាស goto

ប្រកាស goto ត្រូវបានប្រើដើម្បីលោតទៅកន្លែងជាក់លាក់នៅក្នុងកូដ។

```
#include <stdio.h>

int main() {
    int a = 10;
    if (a > 5) {
        goto print_message; // លោតទៅកន្លែង print_message
    }
    printf("This will not be printed");
    print_message:
        printf("a is greater than 5");
    return 0;
}
```

ការពន្យល់៖
- កម្មវិធីនេះប្រើ goto ដើម្បីលោតទៅកន្លែង `print_message` នៅពេល `a` ធំជាង 5។
- ដូច្នេះ, វានឹងបង្ហាញ "a is greater than 5"។

```
Output: a is greater than 5
```

4.8. រចនាសម្ព័ន្ធគ្រប់គ្រងដែលជាន់គ្នា

រចនាសម្ព័ន្ធគ្រប់គ្រងដែលជាន់គ្នាគឺជាការប្រើប្រាស់រចនាសម្ព័ន្ធគ្រប់គ្រងនៅក្នុងរចនាសម្ព័ន្ធគ្រប់គ្រងផ្សេងទៀត។

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    if (a > 5) {
        if (b > 2) {
            printf("Both conditions are true");
        } else {
            printf("Only the first condition is true");
        }
    } else {
        printf("The first condition is false");
    }
    return 0;
}
```

ការពន្យល់៖

- កម្មវិធីនេះប្រើ `if-else` ដែលជាន់គ្នា។
- ប្រសិនបើ a ធំជាង 5 និង b ធំជាង 2, វានឹងបង្ហាញ "Both conditions are true"។
- ប្រសិនបើ a ធំជាង 5 ប៉ុន្តែ b មិនធំជាង 2, វានឹងបង្ហាញ "Only the first condition is true"។
- បើមិនដូច្នេះទេ, វានឹងបង្ហាញ "The first condition is false"។

```
Output: Both conditions are true
```

5. Loops in C

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីប្រភេទនៃលូបនៅក្នុងភាសា C។

5.1. លូប for

លូប `for` ត្រូវបានប្រើដើម្បីធ្វើការកំណត់ចំនួននៃការអនុវត្តន៍ជាក់លាក់។ វាអាចកំណត់ចំនួនកំណត់នៃការងារដែលត្រូវអនុវត្ត។

```c
#include <stdio.h>

int main() {
```

```c
    for (int i = 0; i < 5; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

```
Output: 0
1
2
3
4
```

5.2. លូប while

លូប while ត្រូវបានប្រើដើម្បីអនុវត្តកូដឡើងវិញជាការផ្សេងៗទៅរហូតដល់លក្ខខណ្ឌមួយត្រូវបានបង្ហាញថាត្រឹមត្រូវ។

```c
#include <stdio.h>

int main() {
    int i = 0;
    while (i < 5) {
        printf("%d\n", i);
        i++;
    }
    return 0;
}
```

```
Output: 0
1
2
3
4
```

5.3. លូប do-while

លូប do-while ត្រូវបានប្រើដើម្បីអនុវត្តកូដមួយដំបូងប៉ុន្តែការត្រួតពិនិត្យលក្ខខណ្ឌត្រូវបានអនុវត្តក្រោយពីការអនុវត្តចុងក្រោយ។

```c
#include <stdio.h>

int main() {
    int i = 0;
    do {
        printf("%d\n", i);
        i++;
    } while (i < 5);
    return 0;
}
```

```
Output: 0
1
2
3
4
```

5.4. លូប Nested

លូប Nested ត្រូវបានប្រើដើម្បីបញ្ចូលលូបនៅក្នុងលូប។ វាមានប្រយោជន៍ក្នុងករណីដែលត្រូវការត្រួតពិនិត្យជាច្រើននៅក្នុងវគ្គប្រតិបត្តិការលូប។

```c
#include <stdio.h>

int main() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d%d ", i, j);
        }
    }
    return 0;
}
```

```
Output: 00 01 02 10 11 12 20 21 22
```

5.5. លូបអន្តរក្រោយ

លូបអន្តរក្រោយត្រូវបានប្រើប្រសិនបើលក្ខខណ្ឌមិនត្រូវបានបញ្ចប់។ វាអាចត្រូវបានប្រើប្រាស់ដោយព្រមានដោយស្រប
តាមការប្រើប្រាស់ break សម្រាប់ចាកចេញពីលូប។

```
#include <stdio.h>

int main() {
    while (1) {
        printf("This is an infinite loop\n");
        break; // Exit the loop
    }
    return 0;
}
```

```
Output: This is an infinite loop
```

6. Functions in C

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីមុខងារ (Functions) នៅក្នុងភាសា C។

6.1. ការប្រាប់ និងការបង្កើតមុខងារ

មុខងារ (Function) ត្រូវបានប្រើដើម្បីបែងចែកការងារនៅក្នុងកម្មវិធី។ ការប្រាប់មុខងារមានបញ្ហាទៅក្នុងកូដមុនពី
ការអនុវត្តន៍វា។

```
#include <stdio.h>

// Function declaration
void greet();

int main() {
    greet(); // Function call
    return 0;
}

// Function definition
void greet() {
```

```
    printf("Hello, World!\n");
}
```

ការពន្យល់៖
- មុខងារ greet() ត្រូវបានប្រកាស (declared) មុនពេលអនុវត្តន៍ (defined)។
- ក្នុង main(), មុខងារ greet() ត្រូវបានហៅ (called)។
- លទ្ធផលនឹងបង្ហាញ "Hello, World!"។

```
Output: Hello, World!
```

6.2. ប្រភេទអាគុយម៉ង់ និងប្រភេទត្រឡប់តាមមុខងារ

មុខងារមានអាគុយម៉ង់ ដែលអាចប្រើក្នុងការទទួលយកទិន្នន័យមកក្នុងវា។ ប្រភេទត្រឡប់តាមមុខងារ ក៏អាចធ្វើការត្រឡប់តម្លៃមកវិញបានផងដែរ។

```
#include <stdio.h>

// Function with arguments and return type
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4); // Function call with arguments
    printf("The sum is: %d\n", result);
    return 0;
}
```

ការពន្យល់៖
- មុខងារ add() ទទួលអាគុយម៉ង់ពីរ (a និង b) និងត្រឡប់តម្លៃផលបូក។
- ក្នុង main(), មុខងារ add() ត្រូវបានហៅ ដោយបញ្ជូនតម្លៃ 3 និង 4។
- លទ្ធផលនឹងបង្ហាញ "The sum is: 7"។

```
Output: The sum is: 7
```

6.3. ការហៅដោយខ្លួនឯង

Recursion គឺជាការហៅមុខងារមួយដោយខ្លួនឯង ដើម្បីដោះសោរាបញ្ហាដែលស្រដៀងគ្នា។

```c
#include <stdio.h>

// Recursive function to calculate factorial
int factorial(int n) {
    if (n == 0) {
        return 1; // Base case
    } else {
        return n * factorial(n - 1); // Recursive call
    }
}

int main() {
    int result = factorial(5); // Function call
    printf("Factorial of 5 is: %d\n", result);
    return 0;
}
```

ការពន្យល់៖

- មុខងារ `factorial()` ហៅខ្លួនឯងដើម្បីគណនាហ្វាក់តូរីល។
- លក្ខខណ្ឌបញ្ចប់ (base case) គឺនៅពេល `n == 0`, វាត្រឡប់ 1។
- លទ្ធផលនឹងបង្ហាញ "Factorial of 5 is: 120"។

```
Output: Factorial of 5 is: 120
```

6.4. ការហៅតាមតម្លៃ និងការហៅតាមយោង

ការហៅតាមតម្លៃ (Call by Value) និងការហៅតាមយោង (Call by Reference) មានភាពខុសគ្នាក្នុងការផ្លាស់ប្តូរទិន្នន័យនៅក្នុងមុខងារ។

```c
#include <stdio.h>

// Call by Value
void byValue(int a) {
    a = 20; // Only modifies the local copy
}

// Call by Reference
void byReference(int *a) {
    *a = 20; // Modifies the original value
```

```c
}

int main() {
    int x = 10;
    byValue(x);
    printf("Value after byValue: %d\n", x); // 10

    byReference(&x);
    printf("Value after byReference: %d\n", x); // 20
    return 0;
}
```

ការពន្យល់៖
- byValue() មិនផ្លាស់ប្តូរតម្លៃដើមនៃអថេរ x។
- byReference() ផ្លាស់ប្តូរតម្លៃដើមនៃអថេរ x តាមរយៈអាសយដ្ឋាន។
- លទ្ធផលនឹងបង្ហាញ "Value after byValue: 10" និង "Value after byReference: 20"។

```
Output:
Value after byValue: 10
Value after byReference: 20
```

6.5. ប្រភេទមុខងារ

ប្រភេទមុខងារ (Function Prototypes) ត្រូវបានប្រើដើម្បីស្នើរសុំការប្រាប់ពីបែបបទនៃមុខងារ មុនការអនុវត្តន៍វា។

```c
#include <stdio.h>

// Function prototype
void greet();

int main() {
    greet(); // Function call
    return 0;
}

// Function definition
void greet() {
```

```
    printf("Hello, World!\n");
}
```

ការពន្យល់៖

- ប្រភេទមុខងារ (prototype) `void greet();` ត្រូវបានប្រកាសមុន `main()`។
- ការប្រកាសនេះអនុញ្ញាតឱ្យកម្មវិធីដឹងពីអត្ថិភាពនៃមុខងារ `greet()`។
- លទ្ធផលនឹងបង្ហាញ "Hello, World!"។

```
Output: Hello, World!
```

6.6. កំណត់ឯកសារ និងបណ្ណាល័យ

ឯកសារ header និងបណ្ណាល័យ (Libraries) ត្រូវបានប្រើដើម្បីផ្តល់សេវាកម្មនិងមុខងារផ្សេងៗដែលអាចអនុវត្តបាន។

```
#include <stdio.h>  // Standard library for input-output functions
#include <math.h>   // Library for mathematical functions

int main() {
    printf("Square root of 16 is: %.2f\n", sqrt(16)); // sqrt() function
    return 0;
}
```

ការពន្យល់៖

- `#include <stdio.h>` ប្រើសម្រាប់មុខងារបញ្ចូល/បញ្ចេញ។
- `#include <math.h>` ប្រើសម្រាប់មុខងារគណិតវិទ្យា។
- លទ្ធផលនឹងបង្ហាញ "Square root of 16 is: 4.00"។

```
Output: Square root of 16 is: 4.00
```

6.7. មុខងារបណ្ណាល័យស្តង់ដារ

មុខងារបណ្ណាល័យស្តង់ដារ (Standard Library Functions) នៅក្នុងភាសា C ផ្តល់ជូននូវមុខងារដែលមានប្រយោជន៍សម្រាប់ការងារផ្សេងៗគ្នា។

```
#include <stdio.h>
#include <string.h> // Library for string functions
```

```
int main() {
    char str[] = "Hello, World!";
    printf("Length of string: %lu\n", strlen(str)); // strlen() function
    return 0;
}
```

ការពន្យល់៖

- #include <string.h> ប្រើសម្រាប់មុខងារដែលទាក់ទងនឹងខ្សែអក្សរ។
- strlen() គណនាប្រវែងខ្សែអក្សរ។
- លទ្ធផលនឹងបង្ហាញ "Length of string: 13"។

```
Output: Length of string: 13
```

7. Arrays in C

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីអារេ (Arrays) នៅក្នុងភាសា C។

7.1. អារេមួយវិមាត្រ

អារេមួយវិមាត្រគឺជាការប្រមូលគ្នានៃតម្លៃដែលមានប្រភេទដូចគ្នា។ អារេមួយមិតិយ៉ាងសាមញ្ញនឹងមានកូដដូចខាងក្រោម។

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5}; // One-dimensional array
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]); // Access elements using index
    }
    return 0;
}
```

ការពន្យល់៖

- អារេ arr ត្រូវបានប្រកាសជាអារេមួយវិមាត្រដែលមានទំហំ 5។
- តម្លៃនៃអារេត្រូវបានចាប់ផ្តើមជា 5។

- កូដនេះប្រើ for loop ដើម្បីបង្ហាញធាតុនីមួយៗនៃអារេ។
- លទ្ធផលនឹងបង្ហាញ "1 2 3 4 5"។

```
Output: 1 2 3 4 5
```

7.2. អារេពហុវិមាត្រ

អារេពហុវិមាត្រគឺជាអារេដែលមានជម្រើសតិចតួចជាច្រើន (ធម្មតាជាម៉ាត្រិក)។ អារេនេះអាចប្រើបានដើម្បីផ្ទុកទិន្នន័យដូចជា តារាងលេខ។

```c
#include <stdio.h>

int main() {
    int arr[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    }; // 2x3 two-dimensional array
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", arr[i][j]); // Accessing 2D array elements
        }
        printf("\n");
    }
    return 0;
}
```

ការពន្យល់៖
- អារេ arr ត្រូវបានប្រកាសជាអារេពីរវិមាត្រដែលមានទំហំ 2x3។
- តម្លៃនៃអារេត្រូវបានចាប់ផ្តើមជា 3 និង 6។
- កូដនេះប្រើ nested for loop ដើម្បីបង្ហាញធាតុនីមួយៗនៃអារេ។
- លទ្ធផលនឹងបង្ហាញ "1 2 3" និង "4 5 6"។

```
Output:
1 2 3
4 5 6
```

7.3. ការកំណត់អារេ

ការកំណត់អារេគឺជាការបញ្ជាក់តម្លៃដំបូងសម្រាប់អារេ។ អ្នកអាចកំណត់តម្លៃដំបូងបានដោយប្រើសញ្ញាបញ្ជូន `{}`។

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3}; // Array initialized with partial values
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]); // Remaining values will be set to 0
    }
    return 0;
}
```

ការពន្យល់៖

- អារេ arr ត្រូវបានប្រកាសជាអារេមួយវិមាត្រដែលមានទំហំ 5។
- តម្លៃនៃអារេត្រូវបានចាប់ផ្តើមជា 3 ដោយតម្លៃដែលនៅសល់ត្រូវបានកំណត់ជា 0។
- លទ្ធផលនឹងបង្ហាញ "1 2 3 0 0"។

```
Output: 1 2 3 0 0
```

7.4. ការផ្ទេរអារេទៅមុខងារ

អ្នកអាចផ្ទេរអារេទៅមុខងារ ដោយផ្ទេរតែអាសយដ្ឋាននៃអារេ។

```c
#include <stdio.h>

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    printArray(arr, 5); // Passing array to function
    return 0;
}
```

ការពន្យល់៖

- មុខងារ printArray() ទទួលអារេជាអាគុយម៉ង់ និងបង្ហាញធាតុនីមួយៗ។
- ក្នុង main(), អារេ arr ត្រូវបានផ្ទេរទៅមុខងារ printArray()។
- លទ្ធផលនឹងបង្ហាញ "1 2 3 4 5"។

```
Output: 1 2 3 4 5
```

7.5. ការបដិសេធអារេ

ការបដិសេធអារេអាចមានប្រយោជន៍ដើម្បីកែប្រែទិន្នន័យក្នុងអារេ។ ការបដិសេធអារេអាចធ្វើការផ្លាស់ប្តូរ តម្លៃធាតុនៃអារេ និងគ្រប់គ្រងទិន្នន័យដោយប្រើកូដរួមផ្សំ។

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++) {
        arr[i] = arr[i] * 2; // Multiply each element by 2
    }
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]); // Display updated array
    }
    return 0;
}
```

ការពន្យល់៖

- អារេ `arr` ត្រូវបានប្រកាសជាអារេមួយវិមាត្រដែលមានទំហំ 5។
- កូដនេះប្រើ `for` loop ដើម្បីគុណធាតុនីមួយៗនៃអារេដោយ 2។
- លទ្ធផលនឹងបង្ហាញ "2 4 6 8 10"។

```
Output: 2 4 6 8 10
```

8. Pointers in C

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីPointer (Pointers) នៅក្នុងភាសា C។

8.1. ការណែនាំអំពីPointer

Pointer គឺជាអថេរដែលផ្ទុកអាសយដ្ឋាននៃអថេរផ្សេងទៀត។ យើងប្រើPointer ដើម្បីយកអាសយដ្ឋាននៃអថេរមួយ ហើយអាចប្រើវាដើម្បីចូលដំណើរការទិន្នន័យក្នុងអថេរនោះ។

```
#include <stdio.h>

int main() {
    int a = 10;
    int *ptr = &a;  // ptr is a pointer to a
    printf("Value of a: %d", a);
    printf("Value through ptr: %d", *ptr);  // Dereferencing the pointer
    return 0;
}
```

```
Output:
Value of a: 10
Value through ptr: 10
```

8.2. ការប្រកាស និង ការបង្កើតPointer

ក្នុងភាសា C, Pointerត្រូវបានប្រកាសដោយប្រើសញ្ញា *។ អ្នកអាចបង្កើតPointer និងកំណត់អាសយដ្ឋានឱ្យវាបានដោយប្រើសញ្ញា & ។

```
#include <stdio.h>

int main() {
    int a = 5;
    int *ptr = &a;  // Pointer initialization with address of a
    printf("Address of a: %p", (void*)ptr);  // Print address
    return 0;
}
```

```
Output:
Address of a:
```

8.3. Pointer និង អារេ

Pointerនឹងអារេគឺជាគ្រឿងភ្ជាប់គ្នាដោយពិតប្រាកដ។ យើងអាចប្រើPointer ដើម្បីចូលទៅកាន់ធាតុក្នុងអារេ។

```c
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};
    int *ptr = arr;  // Pointer points to the first element of arr
    for (int i = 0; i < 3; i++) {
        printf("%d ", *(ptr + i));  // Dereference pointer to access arr
    }
    return 0;
}
```

```
Output: 10 20 30
```

8.4. Pointer ទៅមុខងារ

អ្នកអាចប្រើPointer ដើម្បីចូលទៅកាន់មុខងារ និងហៅមុខងារនោះតាមអាសយដ្ឋាន។

```c
#include <stdio.h>

void greet() {
    printf("Hello, World!");
}

int main() {
    void (*ptr)() = greet;  // Pointer to function greet
    ptr();  // Call function using pointer
    return 0;
}
```

```
Output:
Hello, World!
```

8.5. ការបញ្ជូនសម្ភារៈស្មើ

ការបញ្ជូនសម្ភារៈស្មើមានប្រយោជន៍នៅពេលដែលអ្នកត្រូវការបង្កើតអត្ថបទមួយដែលមានទំហំមិនប្រាកដក្នុងរយៈពេលបណ្ដោះអាសន្ន។

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));  // Allocate memory for 5
    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;  // Initialize dynamically allocated memory
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);  // Print dynamically allocated array
    }

    free(ptr);  // Free allocated memory
    return 0;
}
```

```
Output:
1 2 3 4 5
```

8.6. គណិតវិទ្យាPointer

Pointerអាចប្រើសម្រាប់ធ្វើការគណិតវិទ្យា (arithmetics) ដើម្បីបញ្ជូនទៅឬបញ្ចេញនូវអាសយដ្ឋាននៃអថេរផ្សេងៗ។

```c
#include <stdio.h>

int main() {
    int arr[3] = {1, 2, 3};
    int *ptr = arr;
    printf("%d ", *(ptr + 1));  // Pointer arithmetic to access second e
    return 0;
}
```

```
Output:
2
```

8.7. Pointer ទៅStructures និង អារេ

អ្នកអាចប្រើPointer ដើម្បីចូលទៅកាន់ប្រភេទទិន្នន័យស្មុគស្មាញដូចជា structures និងអារេ។

```
#include <stdio.h>

struct Person {
    char name[50];
    int age;
};

int main() {
    struct Person p1 = {"John", 30};
    struct Person *ptr = &p1;
    printf("Name: %s, Age: %d", ptr->name, ptr->age);  // Access struct
    return 0;
}
```

```
Output:
Name: John, Age: 30
```

9. Structures and Unions

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីស្រទាប់ និងអង្គចងចាំ (Structures and Unions) នៅក្នុងភាសា C។

9.1. ការណែនាំអំពីស្រទាប់

ស្រទាប់ (Structures) គឺជាការបញ្ចូលតម្លៃមួយចំនួនជាភាគរយ ក្នុងអង្គតម្លៃផ្សេងៗ។ អ្នកអាចប្រើស្រទាប់ដើម្បីធ្វើការរួមបញ្ចូលទិន្នន័យដែលមានប្រភេទផ្សេងៗគ្នា។

```
#include <stdio.h>

struct Person {
    char name[50];
```

```c
    int age;
};

int main() {
    struct Person person1 = {"John", 30};
    printf("Name: %s", person1.name);
    printf("Age: %d", person1.age);
    return 0;
}
```

```
Output:
Name: John
Age: 30
```

9.2. ការបញ្ចូលនិងកំណត់តម្លៃស្រទាប់

ការបញ្ចូលស្រទាប់ជាដើម និងការកំណត់តម្លៃប្រើរបៀបដូចខាងក្រោម។

```c
#include <stdio.h>

struct Student {
    char name[50];
    int rollNumber;
    float marks;
};

int main() {
    struct Student student1 = {"Alice", 101, 90.5};
    printf("Name: %s", student1.name);
    printf("Roll Number: %d", student1.rollNumber);
    printf("Marks: %.2f", student1.marks);
    return 0;
}
```

```
Output:
Name: Alice
Roll Number: 101
Marks: 90.50
```

9.3. ការចូលប្រើសមាជិកស្រទាប់

ដើម្បីចូលប្រើសមាជិកស្រទាប់ អ្នកត្រូវប្រើសញ្ញាសម្គាល់ហធិបតេយ្យ `.` ។

```
#include <stdio.h>

struct Person {
    char name[50];
    int age;
};

int main() {
    struct Person person1 = {"John", 30};
    printf("Name: %s", person1.name);  // Accessing structure member
    printf("Age: %d", person1.age);
    return 0;
}
```

```
Output:
Name: John
Age: 30
```

9.4. ស្រទាប់ជាប់គ្នា

អ្នកអាចប្រើស្រទាប់នៅក្នុងស្រទាប់ផ្សេងទៀត ដើម្បីផ្ទុកព័ត៌មានបន្ថែម។ នេះអាចធ្វើបានតាមរយៈការបញ្ចូលរបស់ស្រទាប់ដទៃ។

```
#include <stdio.h>

struct Date {
    int day;
    int month;
    int year;
};

struct Person {
    char name[50];
    struct Date birthDate;  // Nested structure
};
```

```
int main() {
    struct Person person1 = {"John", {15, 5, 1990}};
    printf("Name: %s", person1.name);
    printf("Birth Date: %d/%d/%d", person1.birthDate.day, person1.birthD
    return 0;
}
```

```
Output:
Name: John
Birth Date: 15/5/1990
```

9.5. អង្គចងចាំក្នុង C

អង្គចងចាំ (Union) គឺជាដំណើរការដែលអនុញ្ញាតឲ្យមានការប្រើប្រាស់ប្រភេទទិន្នន័យជាច្រើនក្នុងមួយផ្នែកតែម្តង។ តែការប្រើប្រាស់ទិន្នន័យជាច្រើនប្រភេទក្នុងចំណោមវា គួរតែមានទំហំតែមួយ។

```
#include <stdio.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;
    data.i = 10;
    printf("Data as integer: %d", data.i);
    data.f = 220.5;
    printf("Data as float: %.2f", data.f);
    return 0;
}
```

```
Output:
Data as integer: 10
Data as float: 220.50
```

9.6. ការប្រៀបធៀបរវាងស្រទាប់ និងអង្គចងចាំ

មានការប្រៀបធៀបមួយចំនួនរវាងស្រទាប់ និងអង្គចងចាំ។ ភាពខុសគ្នាដ៏សំខាន់គឺថាស្រទាប់ត្រូវបានប្រើដើម្បីផ្ទុកទិន្នន័យពីប្រភេទផ្សេងៗគ្នា ហើយទាំងអស់មានតម្លៃដែលត្រូវបានរក្សាទុកជាដំណាក់កាល។ តែអង្គចងចាំ គឺស្តាប់តែប្រភេទទិន្នន័យមួយគត់ នៅពេលនោះ។

10. File Handling

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីការដោះសោទិងការប្រតិបត្តិការពីឯកសារ (File Handling) នៅក្នុងភាសា C។

10.1. ការណែនាំអំពីការដោះសោឯកសារ

File Handling គឺជាការប្រើប្រាស់ឯកសារជាមួយភាសា C ដើម្បីអាន បញ្ចូល ឬលុបព័ត៌មានពីឯកសារ។ ឯកសារនៅក្នុង C គឺអាចសរសេរ និងអានបាន។

```c
#include <stdio.h>

int main() {
    FILE *file = fopen("file.txt", "w");  // Open a file for writing
    if (file != NULL) {
        fprintf(file, "Hello, world!");  // Write to the file
        fclose(file);  // Close the file
    }
    return 0;
}
```

```
Output: (Creates a file named "file.txt" with the content "Hello, world!
```

10.2. ប្រតិបត្តិការនៅលើឯកសារ: fopen, fclose, fread, fwrite

ភាសា C ផ្តល់នូវមុខងារផ្សេងៗដើម្បីអាន និងសរសេរ ទិន្នន័យទៅក្នុងឯកសារ។ ការប្រើប្រាស់មុខងារ fopen() និង fclose() សម្រាប់បើក និងបិទឯកសារ និង fread(), fwrite() សម្រាប់អាន និងសរសេរ ទិន្នន័យ។

```c
#include <stdio.h>

int main() {
    FILE *file = fopen("file.txt", "r");  // Open file for reading
    char str[100];
    if (file != NULL) {
        fread(str, sizeof(char), 100, file);  // Read from the file
        printf("Read content: %s", str);
        fclose(file);  // Close the file
    }
    return 0;
}
```

```
Output:
Read content: Hello, world!
```

10.3. អាសយដ្ឋានឯកសារ និងមូដ

អ្នកអាចប្រើមុខងារ fopen() ដើម្បីបើកឯកសារជាមួយមូដផ្សេងៗ។ មូដទាំងនេះអាចជាផ្នែកអាន ('r'), សរសេរ ('w'), ឬបន្ថែម ('a') ទៅក្នុងឯកសារ។

```c
#include <stdio.h>

int main() {
    FILE *file = fopen("file.txt", "w");  // Open file in write mode
    if (file != NULL) {
        fprintf(file, "This is a test.");
        fclose(file);  // Close the file
    }
    return 0;
}
```

```
Output: (Creates or overwrites "file.txt" with the text "This is a test.
```

10.4. អាន និងសរសេរឯកសារប្រើអក្សរ

ការអាន និងសរសេរឯកសារដោយប្រើអក្សរនៅក្នុង C ត្រូវបានធ្វើដោយប្រើមុខងារ fopen(), fwrite(), fread() និង fclose()។

```
#include <stdio.h>

int main() {
    FILE *file = fopen("file.txt", "r");  // Open file in read mode
    char ch;
    if (file != NULL) {
        while ((ch = fgetc(file)) != EOF) {  // Read character by charac
            putchar(ch);
        }
        fclose(file);  // Close the file
    }
    return 0;
}
```

```
Output:
Content of the file displayed character by character
```

10.5. ការដោះសោនកំហុសក្នុងប្រតិបត្តិការឯកសារ

ការដោះសោនកំហុសនៅក្នុងប្រតិបត្តិការឯកសារអាចធ្វើបានដោយប្រើមុខងារដូចជា `fopen()` ដែលត្រឡប់តែបែបតាមឯកសារបើកបានឬមិនបាន និងផ្តល់ព័ត៌មានបន្ថែមពាក់ព័ន្ធក្នុងករណីមិនអាចបើកឯកសារ។

```
#include <stdio.h>

int main() {
    FILE *file = fopen("nonexistent.txt", "r");  // Try to open a non-ex
    if (file == NULL) {
        printf("Error: Could not open file.");
    } else {
        fclose(file);
    }
    return 0;
}
```

```
Output:
Error: Could not open file.
```

10.6. ការដោះសោឯកសារប្រភេទបាញ់ (Binary Files)

ការប្រើប្រាស់ឯកសារប្រភេទបាញ់ (Binary) អាចធ្វើបានដោយប្រើមុខងារ fopen() ដោយប្រើម៉ូដ "wb" សម្រាប់សរសេរ និង "rb" សម្រាប់អាន។

```c
#include <stdio.h>

int main() {
    FILE *file = fopen("data.bin", "wb");  // Open binary file for writi
    int num = 1234;
    fwrite(&num, sizeof(num), 1, file);  // Write binary data to file
    fclose(file);  // Close the file

    file = fopen("data.bin", "rb");  // Open binary file for reading
    fread(&num, sizeof(num), 1, file);  // Read binary data from file
    printf("Read number: %d", num);
    fclose(file);
    return 0;
}
```

```
Output:
Read number: 1234
```

11. Dynamic Memory Allocation

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីការបែងចែកសម្ភារៈនៅលើចន្លោះម៉ាស៊ីនយ៉ាងឥតកំណត់ (Dynamic Memory Allocation) នៅក្នុងភាសា C។

11.1. ការណែនាំអំពីសម្ភារៈឥតកំណត់

Dynamic Memory Allocation គឺជាការបែងចែកអង្គចិត្តនៃសម្ភារៈមួយនៅពេលកម្មវិធីកំពុងដំណើរការដោយប្រើមុខងារ malloc(), calloc(), realloc() និង free()។

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    ptr = (int*) malloc(sizeof(int));  // Allocate memory for one intege
    if (ptr != NULL) {
        *ptr = 100;
        printf("Value: %d", *ptr);
        free(ptr);  // Free dynamically allocated memory
    }
    return 0;
}
```

```
Output:
Value: 100
```

11.2. malloc(), calloc(), realloc(), and free()

មុខងារ malloc() និង calloc() ត្រូវបានប្រើដើម្បីបែងចែកអង្គចិត្តម៉ាស៊ីនសម្រាប់ទិន្នន័យដែលមិនបានកំណត់ពីមុន (Dynamic Memory). realloc() ត្រូវបានប្រើដើម្បីបន្ថែមឬកាត់បន្ថយទំហំមាតិកាដែលបានបែងចែក។ free() ប្រើសម្រាប់ដោះលែងអង្គចិត្តដែលបានបែងចែក។

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;
    arr = (int*) malloc(5 * sizeof(int));  // Allocate memory for 5 inte
    if (arr != NULL) {
        for (int i = 0; i < 5; i++) {
            arr[i] = i * 10;
        }

        for (int i = 0; i < 5; i++) {
```

```
            printf("%d ", arr[i]);  // Display elements
        }

        arr = (int*) realloc(arr, 10 * sizeof(int));  // Reallocate memo
        for (int i = 5; i < 10; i++) {
            arr[i] = i * 10;
        }

        printf(" ");
        for (int i = 0; i < 10; i++) {
            printf("%d ", arr[i]);  // Display updated elements
        }

        free(arr);  // Free allocated memory
    }
    return 0;
}
```

```
Output:
0 10 20 30 40
0 10 20 30 40 50 60 70 80 90
```

11.3. ការហួសបាត់នៃសម្ភារៈ និងការគ្រប់គ្រងសម្ភារៈ

Memory Leaks គឺជាបញ្ហានៃការបែងចែកសម្ភារៈហើយមិនមានការដោះលែងមើលថែរប្រាកដ។ វាអាចបណ្តាលឱ្យមានបញ្ហាកំណត់ក្នុងប្រព័ន្ធ។ ការគ្រប់គ្រងសម្ភារៈប្រកបដោយការប្រើប្រាស់ free() និងការបង់កំណត់លើការបែងចែកមានសារៈសំខាន់។

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*) malloc(5 * sizeof(int));  // Memory allocated but
    // Forgetting to call free() can cause a memory leak
    // free(arr);  // Fix by freeing the allocated memory
    return 0;
}
```

```
Output:
(No output, but this program has a memory leak due to not freeing the me
```

11.4. ការប្រើប្រាស់បញ្ជូនជាមួយសម្ភារៈឥតកំណត់

ការប្រើប្រាស់ pointers ជាមួយនឹង dynamic memory គឺមានសារៈសំខាន់ពេលដែលអ្នកកំណត់តម្លៃឥតកំណត់មួយសម្រាប់ទិន្នន័យ។ ការប្រើប្រាស់ pointer អាចធ្វើឲ្យការបែងចែក memory ទៅលើអង្គចិត្តនីមួយៗកាន់តែមានប្រសិទ្ធភាព។

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    ptr = (int*) malloc(3 * sizeof(int));  // Allocate memory for 3 inte
    if (ptr != NULL) {
        for (int i = 0; i < 3; i++) {
            ptr[i] = i * 5;  // Assign values dynamically
        }

        for (int i = 0; i < 3; i++) {
            printf("%d ", ptr[i]);  // Display values
        }

        free(ptr);  // Free allocated memory
    }
    return 0;
}
```

```
Output:
0 5 10
```

12. Preprocessor Directives

នៅក្នុងផ្នែកនេះ យើងនឹងសិក្សាពីការប្រើប្រាស់បញ្ជាតម្រូវ (Preprocessor Directives) ក្នុងភាសា C។

12.1. #define and Constants

Directives #define ត្រូវបានប្រើសម្រាប់ការបង្កើត constants និងកំណត់តម្លៃថេរដែលត្រូវបានប្រើក្នុងកម្មវិធី។

```
#include <stdio.h>

#define PI 3.14  // Defining constant PI

int main() {
    float area = PI * 10 * 10;  // Using defined constant
    printf("Area of circle: %.2f", area);
    return 0;
}
```

```
Output:
Area of circle: 314.00
```

12.2. #include and Header Files

Directive #include ត្រូវបានប្រើដើម្បីបញ្ចូលឯកសារ header ដែលមានប្រភេទនៃអនុបត្តម្មដូចជា function declarations, constants, and types។

```
#include <stdio.h>  // Standard Input-Output header file

int main() {
    printf("Hello, World!");  // Using printf from stdio.h
    return 0;
}
```

```
Output:
Hello, World!
```

12.3. Conditional Compilation

Conditional Compilation គឺជាការប្រើប្រាស់បញ្ញាតម្រូវ #if, #else, #elif និង #endif ដើម្បីជ្រើសរើសកូដមួយសម្រាប់បរិស្ថានផ្សេងៗគ្នា។

```
#include <stdio.h>

#define DEBUG 1  // Enable debug code

int main() {
    #if DEBUG
        printf("Debugging is enabled.");
    #else
        printf("Debugging is disabled.");
    #endif
    return 0;
}
```

```
Output:
Debugging is enabled.
```

12.4. Macros and Inline Functions

Macros ត្រូវបានប្រើសម្រាប់បង្កើតកូដផ្នែកក្នុង (code snippets) ដែលត្រូវបានជំនួសដោយអត្ថបទមុននឹងកំណត់ដំណើរការ។ Inline functions គឺជាមុខងារដែលត្រូវបានកំណត់ក្នុងផ្ទាំងមួយនៃការជួបប្រទះដោយមិនបង្កើតការហៅមុខងារ។

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))  // Macro for square of a number

inline int square(int x) {
    return x * x;  // Inline function for square of a number
}

int main() {
    printf("Square using macro: %d", SQUARE(4));
    printf("Square using inline function: %d", square(4));
    return 0;
}
```

```
Output:
Square using macro: 16
Square using inline function: 16
```

12.5. File Inclusion and Guard Directives

Guard directives (e.g., #ifndef, #define, #endif) ត្រូវបានប្រើដើម្បីការពារការបញ្ចូលឯកសារតែម្តងក្នុងកម្មវិធីជាមួយនឹងការជៀសវាងភាពបញ្ចូលអំពីរបៀបមួយ។

```c
// Example of header guard
#ifndef HEADER_FILE_H
#define HEADER_FILE_H

void exampleFunction();

#endif  // HEADER_FILE_H
```

```
Explanation:
This code ensures that the header file is only included once in a projec
```

13. ការដោះសោ និង ការកែចំ

ក្នុងផ្នែកនេះយើងនឹងពិភាក្សាពីបច្ចេកទេសនៃការដោះសោ និង ការកែចំក្នុងការគ្រប់គ្រងកំហុស និងការត្រួតពិនិត្យបញ្ហាក្នុងការសរសេរកូដ C។ ការប្រើប្រាស់ឧបករណ៍ និងបច្ចេកទេសទាំងនេះជួយអ្នកដើម្បីកំណត់ និងដោះសោបញ្ហាដោយឆាប់រហ័សនៅពេលដែលអ្នកកំពុងអភិវឌ្ឍកម្មវិធី។

13.1. ការដោះសោក្នុង C

ក្នុង C ការដោះសោធម្មតាត្រូវបានធ្វើតាមការប្រើតម្លៃត្រឡប់ `errno` និងមុខងារដោះសោដូចជា `perror()` និង `strerror()`។ ឧបករណ៍ទាំងនេះអនុញ្ញាតឱ្យអ្នករកឃើញ និងរាយការណ៍ពីកំហុសនៅពេលដែលកម្មវិធីកំពុងដំណើរការ។

```c
#include <stdio.h>
#include <errno.h>

int main() {
    FILE *file = fopen("non_existent_file.txt", "r");
    if (file == NULL) {
        perror("កំហុសក្នុងការបើកឯកសារ");  // បង្ហាញសារកំហុស
        return 1;
    }
    fclose(file);
    return 0;
}
```

```
Output:
កំហុសក្នុងការបើកឯកសារ: មិនមានឯកសារទេ
```

13.2. ប្រើ assert() សម្រាប់ការកែចំ

មុខងារ `assert()` ជាឧបករណ៍សម្រាប់ត្រួតពិនិត្យលក្ខខណ្ឌមួយ។ ប្រសិនបើលក្ខខណ្ឌមិនត្រឹមត្រូវ ប្រព័ន្ធនឹងបញ្ចប់ការប្រតិបត្តិ ដោយផ្តល់ព័ត៌មានអំពីទីតាំងកំហុសដែលបានកើតឡើង។

```c
#include <stdio.h>
#include <assert.h>

int main() {
    int x = 5;
    assert(x > 0);  // នេះនឹងជោគជ័យ
    assert(x < 0);  // នេះនឹងបរាជ័យ
    return 0;
}
```

```
Output:
ការបញ្ចេញអនុញ្ញាតបានបរាជ័យ: x < 0, ឯកសារអ្នកឧទាហរណ៍.c, បន្ទាត់ 6, មុខងារ m
```

13.3. ការដោះសោកំហុសនៅពេលប្រតិបត្តិ

កំហុស Runtime ក្នុងកម្មវិធី C អាចដោះសោបានតាមបច្ចេកទេសដូចជា ការត្រួតពិនិត្យ pointer មិនមានតម្លៃ, ការបញ្ចាក់រាលបញ្ចូល និងការប្រើប្រាស់លេខកូដកំហុសដែលត្រឹមត្រូវសម្រាប់មុខងារផ្សេងៗ។

```
#include <stdio.h>

int main() {
    int *ptr = NULL;
    if (ptr == NULL) {
        printf("Pointer មិនមានតម្លៃ, មិនអាចបញ្ចូលបាន");
    }
    return 0;
}
```

```
Output:
Pointer មិនមានតម្លៃ, មិនអាចបញ្ចូលបាន
```

13.4. ការកែចំជាមួយ GDB

GDB (GNU Debugger) គឺជាឧបករណ៍ដ៏មានអំណាចសម្រាប់ការកែចំកម្មវិធី C។ វាអនុញ្ញាតឱ្យអ្នកអនុវត្តកម្មវិធីរបស់អ្នកជាការប្រតិបត្តិទីៗ កំណត់ breakpoints ពិនិត្យតម្លៃអថេរ និងតាមដានកំហុស។

```
$ gcc -g example.c -o example   // បង្កើតការបញ្ចូលជាមួយសញ្ញារូបសំណើបបញ្ហា
$ gdb ./example                // ចាប់ផ្តើម GDB
(gdb) break main               // កំណត់ breakpoint នៅក្នុងមុខងារ main
(gdb) run                      // ប្រតិបត្តិកម្មវិធី
(gdb) print x                  // ពិនិត្យតម្លៃនៃ x
```

```
Explanation:
GDB អនុញ្ញាតឱ្យអ្នកធ្វើការកែចំកម្មវិធីបានយ៉ាង interactive និងពិនិត្យប្រភេទរបស់វា។
```

13.5. បច្ចេកទេសការដោះសោបញ្ហាអាក្រក់

បញ្ហាអាក្រក់នៅក្នុង C មិនមានការគាំទ្រដោយតែមួយសម្រាប់ការដោះសោបញ្ហាបានទេ។ ទោះជាយ៉ាងណាក៏ដោយ អ្នកអាចប្រើប្រាស់បច្ចេកទេសដូចជា `setjmp()` និង `longjmp()` ដើម្បីធ្វើការទំនាក់ទំនងដូចជា exception handling នៅក្នុង C ។

```
#include <stdio.h>
#include <setjmp.h>

jmp_buf env;

void testFunction() {
    longjmp(env, 1);  // ការត្រឡប់មកវិញទៅកាន់ setjmp
}

int main() {
    if (setjmp(env) != 0) {
        printf("មានកំហុសកើតឡើង!");
    } else {
        testFunction();  // នេះនឹងបង្ហាញ longjmp
    }
    return 0;
}
```

```
Output:
មានកំហុសកើតឡើង!
```

14. គំនិតខុសៗទៅក្នុង C

ក្នុងផ្នែកនេះ យើងនឹងពិភាក្សាអំពីប្រធានបទខុសៗទៀតក្នុងការកម្មវិធី C ដែលសំខាន់សម្រាប់ការបង្កើតកម្មវិធីដែលមានភាពស្មុគស្មាញ និងមានប្រសិទ្ធភាព។ គំនិតទាំងនេះនឹងពង្រឹងការយល់ដឹងមូលដ្ឋានរបស់អ្នក និងជួយដោះសោបញ្ហាស្មុគស្មាញ និងធ្វើឲ្យប្រសើរឡើងនូវក្រុមការងាររបស់អ្នក។

14.1. Multithreading ក្នុង C

Multithreading អនុញ្ញាតឱ្យមានការអនុវត្តន៍threads ច្រើន (គ្រឿងប្រតិបត្តិតូចៗនៃដំណើរការមួយ) ក្នុងពេលតែមួយ ដែលធ្វើអោយកម្មវិធីមានប្រសិទ្ធភាពកាន់តែខ្ពស់ក្នុងករណីដែលត្រូវការអនុវត្តន៍ពេលតែមួយ ដូចជា កម្មវិធីម៉ាស៊ីនមេ ឬកម្មវិធីដែលត្រូវការអនុវត្តន៍ជាច្រើន។

```
#include <stdio.h>
#include <pthread.h>
```

```c
void* printMessage(void* ptr) {
    printf("សួស្តីពីthread!\n");
    return NULL;
}

int main() {
    pthread_t thread1;
    pthread_create(&thread1, NULL, printMessage, NULL);
    pthread_join(thread1, NULL);
    return 0;
}
```

```
Output:
សួស្តីពីthread!
```

14.2. សោនិងបណ្ដាញក្នុង C

Sockets ផ្ដល់នូវវិធីសាស្ត្រដើម្បីកម្មវិធីធ្វើការទំនាក់ទំនងតាមបណ្ដាញ។ ក្នុង C ប្រើបណ្ណាល័យ <sys/socket.h> ដើម្បីបង្កើតសោនិងធ្វើការទំនាក់ទំនងទាំងពីរសម្រាប់គ្រឿងម៉ាស៊ីនមេនិងអតិថិជន។ វាជាមុខងារសំខាន់សម្រាប់បង្កើតកម្មវិធីដែលត្រូវការបញ្ជូនទិន្នន័យ ដូចជា ប្រព័ន្ធប្រតិបត្តិការទំព័របណ្ដាញ ឬហ្គេមអនឡាញ។

```c
#include <stdio.h>
#include <sys/socket.h>
#include <arpa/inet.h>

int main() {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in server;
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(sockfd, (struct sockaddr *)&server, sizeof(server));
    // បញ្ជូន និងទទួលទិន្នន័យ

    return 0;
}
```

```
Explanation:
នេះបង្កើតគ klient TCP មួយដែលភ្ជាប់ទៅម៉ាស៊ីនមេនៅលើ localhost នៅពេល 8080។
```

14.3. C និងការកម្មវិធីមូលដ្ឋានវត្ថុ-oriented

C មិនមែនជាភាសាកម្មវិធីមូលដ្ឋានវត្ថុទេ ប៉ុន្តែគម្រោងវត្ថុ-oriented ដូចជា encapsulation, inheritance និង polymorphism អាចត្រូវបានអនុវត្តន៍តាមរយៈស៊ិចថូរ (structures), function pointers និងបច្ចេកទេសផ្សេងៗ។ នេះអនុញ្ញាតឱ្យអ្នកសម្រួលទិន្នន័យក្នុងរបៀបដូចកម្មវិធីមូលដ្ឋានវត្ថុ។

```c
#include <stdio.h>

struct Car {
    char model[50];
    int year;
    void (*displayInfo)(struct Car*);  // Function pointer
};

void displayCarInfo(struct Car* car) {
    printf("ម៉ូដែល: %s\nឆ្នាំ: %d\n", car->model, car->year);
}

int main() {
    struct Car myCar = {"Toyota", 2020, displayCarInfo};
    myCar.displayInfo(&myCar);  // ការហៅវិធី
    return 0;}
```

```
Output:
ម៉ូដែល: Toyota
ឆ្នាំ: 2020
```

14.4. រចនាសម្ព័ន្ធទិន្នន័យខ្ពស់ (ដូចជា ដើមឈើ, សៀវភៅ)

រចនាសម្ព័ន្ធទិន្នន័យខ្ពស់ដូចជា ដើមឈើ និង សៀវភៅមានសារៈសំខាន់សម្រាប់ការកំណត់ទិន្នន័យស្មុគស្មាញក្នុង C។ រចនាសម្ព័ន្ធទាំងនេះអនុញ្ញាតឱ្យមានការកែប្រែទិន្នន័យយ៉ាងមានប្រសិទ្ធភាព និងដោះសោបញ្ហាក្នុងកម្មវិធីដូចជា ឌីតាបេស, កម្មវិធីបកប្រែ និង អាល់ហ្គូរិធម៍ AI។

```c
#include <stdio.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

void inOrderTraversal(struct Node* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->data);
        inOrderTraversal(root->right);
    }
}

int main() {
    struct Node root = {10, NULL, NULL};
    struct Node leftChild = {5, NULL, NULL};
    struct Node rightChild = {15, NULL, NULL};

    root.left = &leftChild;
    root.right = &rightChild;

    inOrderTraversal(&root);  // Output: 5 10 15
    return 0;}
```

```
Output:
5 10 15
```

14.5. បច្ចេកទេសប្រសិទ្ធភាពក្រុមបកប្រែ

បច្ចេកទេសប្រសិទ្ធភាពក្រុមបកប្រែមកនឹងត្រូវបានប្រើដើម្បីធ្វើឲ្យកម្មវិធី C មានប្រសិទ្ធភាពខ្ពស់។ បច្ចេកទេសទាំងនេះអាចបង្កើតឲ្យមានការកំណត់ទាំងការចំណាយពេលវេលា ការប្រើប្រាស់លុបចោលកូដដែលមិនចាំបាច់ ការកំណត់អនុភាពនៅក្នុងរយៈពេល និងកំណត់ការប្រើប្រាស់មេម៉ូរី។

```
$ gcc -O2 program.c -o program  // បកប្រែជាមួយកម្រិតប្រសិទ្ធភាព 2
```

```
Explanation:
ផ្កាំង `-02` អនុញ្ញាតឱ្យមានការបង្កើតប្រសិទ្ធភាពដែលធ្វើឲ្យប្រសើរឡើងដោយគ្មានការកំណត់
```

15. បណ្ណាល័យស្តង់ដារ C

ក្នុងផ្នែកនេះ យើងនឹងពិភាក្សាអំពីបណ្ណាល័យស្តង់ដារបស់ភាសា C ដែលផ្តល់អំពីមុខងារនិងសមត្ថភាពដែលអាចជួយសម្រួលការអភិវឌ្ឍកម្មវិធីរបស់អ្នក។ វាជារបៀបដ៏សំខាន់ក្នុងការបង្កើតកម្មវិធីដែលមានប្រសិទ្ធភាព និងគាំទ្រដល់ការអភិវឌ្ឍកម្មវិធីនៅក្នុងបរិស្ថាន C។

15.1. ការបញ្ចូល/ចេញស្តង់ដារ

បណ្ណាល័យ C ផ្តល់នូវមុខងារសំខាន់ៗសម្រាប់ការបញ្ចូល និងចេញទិន្នន័យ៖ `printf()` សម្រាប់បង្ហាញការចេញ និង `scanf()` សម្រាប់ទទួលការបញ្ចូលពីអ្នកប្រើ។

```c
#include <stdio.h>

int main() {
    int number;
    printf("បញ្ចូលលេខ: ");
    scanf("%d", &number);
    printf("លេខដែលបានបញ្ចូលគឺ: %d", number);
    return 0;
}
```

```
Output:
បញ្ចូលលេខ: 5
លេខដែលបានបញ្ចូលគឺ: 5
```

15.2. មុខងារការគ្រប់គ្រងស្ថិតិ

បណ្ណាល័យ C មានមុខងារច្រើនសម្រាប់ការគ្រប់គ្រងស្ថិតិដូចជា `strlen()`, `strcpy()`, `strcat()`, និង `strcmp()`។ មុខងារទាំងនេះអាចធ្វើអោយអ្នកគ្រប់គ្រងអត្ថបទបានល្អប្រសើរឡើង។

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str1[] = "សួស្តី";
    char str2[20];

    strcpy(str2, str1);
    printf("str2: %s", str2);
    printf("ប្រវែងរបស់ str1: %zu", strlen(str1));

    return 0;
}
```

```
Output:
str2: សួស្តី
ប្រវែងរបស់ str1: 6
```

15.3. មុខងារធ្វើគណនាឧត្តម

បណ្ណាល័យ C ផ្តល់នូវមុខងារដើម្បីអនុវត្តន៍ការគណនាឧត្តមដូចជា `sqrt()` សម្រាប់ការគណនាផលដំណាក់ជាកំណត់ និង `pow()` សម្រាប់ការលើកកម្ពស់។

```c
#include <stdio.h>
#include <math.h>

int main() {
    double number = 16.0;
    printf("ការគណនាផលដំណាក់របស់ %f គឺ: %f", number, sqrt(number));
    printf("16 លើកកម្ពស់ 2 គឺ: %f", pow(16, 2));
    return 0;
}
```

```
Output:
ការគណនាផលដំណាក់របស់ 16.000000 គឺ: 4.000000
16 លើកកម្ពស់ 2 គឺ: 256.000000
```

15.4. មុខងារគ្រប់គ្រងមេម៉ូរី

ការគ្រប់គ្រងមេម៉ូរីក្នុង C គឺមានសារៈសំខាន់សម្រាប់ការផ្តល់ដល់កម្មវិធីការចំណាយមេម៉ូរីដែលមានប្រសិទ្ធភាព។ `malloc()`, `free()`, និង `realloc()` ជាមុខងារសំខាន់សម្រាប់ការប្រមូលទិន្នន័យនៅលើមេម៉ូរី។

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int* ptr = (int*)malloc(sizeof(int));
    if (ptr == NULL) {
        printf("មេម៉ូរីមិនគ្រប់គ្រាន់");
        return 1;
    }
    *ptr = 42;
    printf("តម្លៃក្នុង ptr: %d", *ptr);
    free(ptr);  // បញ្ចប់ការប្រើប្រាស់មេម៉ូរី
    return 0;
}
```

```
Output:
តម្លៃក្នុង ptr: 42
```

15.5. មុខងារពេលវេលានិងថ្ងៃ

C ផ្តល់នូវមុខងារច្រើនសម្រាប់ការកំណត់ និងទទួលថ្ងៃ និងពេលវេលាដូចជា `time()`, `localtime()`, និង `strftime()`។ មុខងារទាំងនេះអាចធ្វើអោយអ្នកគ្រប់គ្រងថ្ងៃ និងពេលវេលានៅក្នុងកម្មវិធីបានល្អប្រសើរ។

```
#include <stdio.h>
#include <time.h>

int main() {
    time_t t;
    time(&t);
    struct tm *tm_info = localtime(&t);
    printf("ថ្ងៃបច្ចុប្បន្ន: %s", asctime(tm_info));
    return 0;
}
```

```
Output:
ថ្ងៃបច្ចុប្បន្ន: Mon Jan 15 10:56:31 2025
```

15.6. មុខងារគ្រប់គ្រងតួអក្សរ

C ផ្តល់នូវមុខងារគ្រប់គ្រងតួអក្សរដូចជា `isalpha()`, `isdigit()`, និង `tolower()` ដែលអាចត្រូវបានប្រើសម្រាប់ការត្រួតពិនិត្យ និងផ្លាស់ប្តូរតួអក្សរ។

```c
#include <stdio.h>
#include <ctype.h>

int main() {
    char ch = 'A';
    if (isalpha(ch)) {
        printf("%c គឺជាតួអក្សរ", ch);
    }
    printf("%c នៅក្នុងតួអក្សរតូចគឺ: %c", ch, tolower(ch));
    return 0;
}
```

```
Output:
A គឺជាតួអក្សរ
A នៅក្នុងតួអក្សរតូចគឺ: a
```

អំពីយើង

CodeKhmerLearning ផ្តល់ឱ្យអ្នកនូវវគ្គសិក្សាពិសេសក្នុងការរៀនកូដភាសាខ្មែរ។ យើងមានក្រុមការងារព្រមមានជំនាញក្នុងការអភិវឌ្ឍកម្មវិធី និងរេបសាយ។

© 2025 CodeKhmerLearning. All Rights Reserved.

តំណភ្ជាប់រហ័ស

ផ្ទាំងដើម
វគ្គសិក្សា
អំពីយើង
ទំនាក់ទំនង

ទំនាក់ទំនង

សូមទំនាក់ទំនងមកកាន់យើងក្តី:

Email: thymuoyhak.biu@gmail.com

Phone: +855 888 052 812

© 2025 CodeKhmerLearning. All rights reserved.